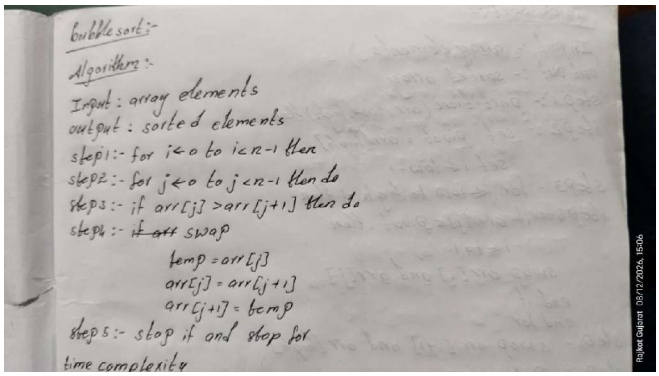
PRACTICAL: 01

AIM :- Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Algorithms:- Bubble sort



Program/code:-

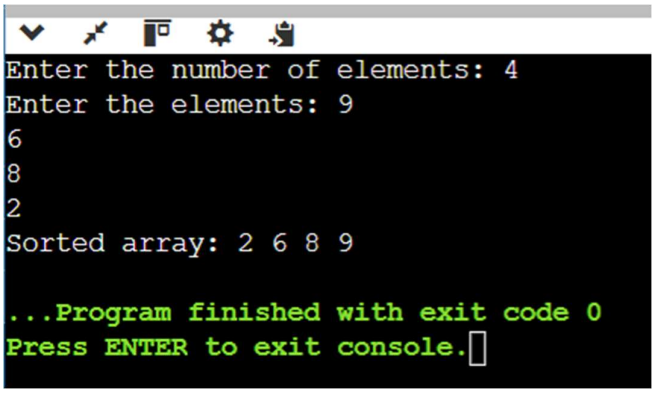
```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
```

```
    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

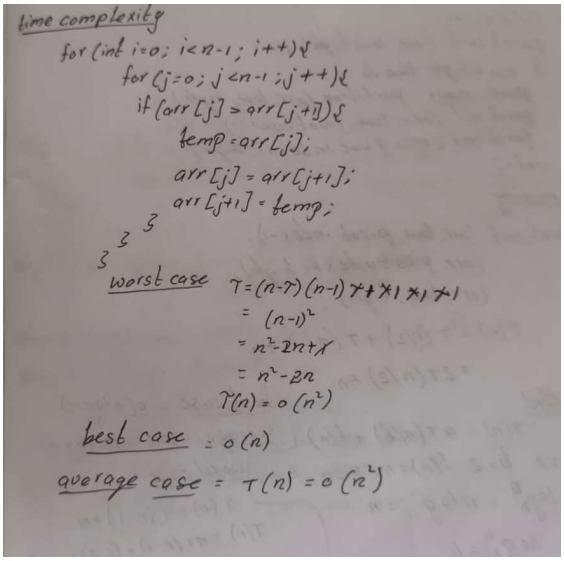
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

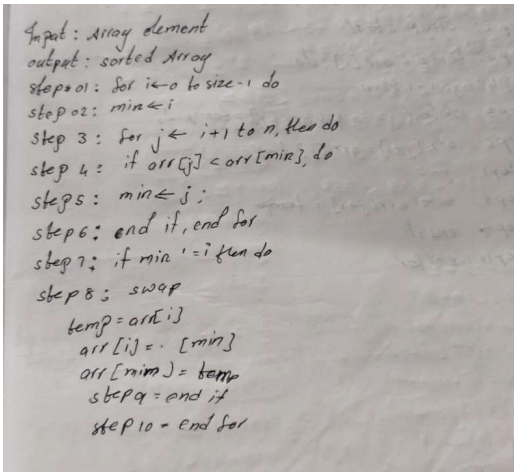
Output:-



Timecomplexity:-



Algorithms:- Selection sort

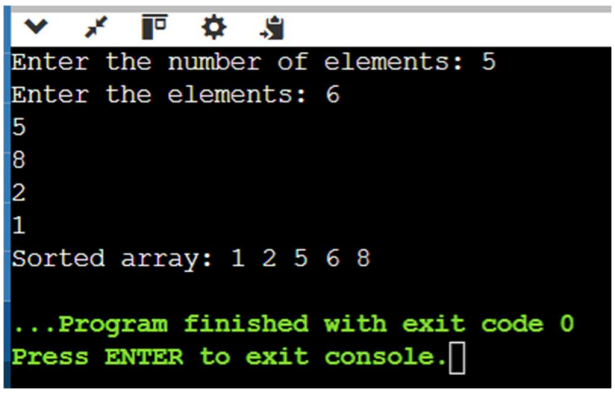


Program/code:-

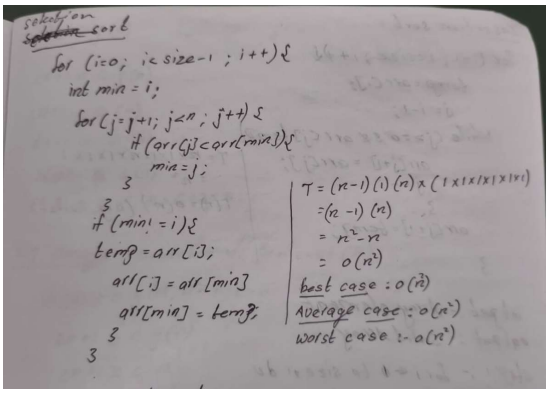
```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    // Selection Sort
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min]) {
                min = j;
            }
        }
        // Swap
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
```

```
    }
    return 0;
}
```

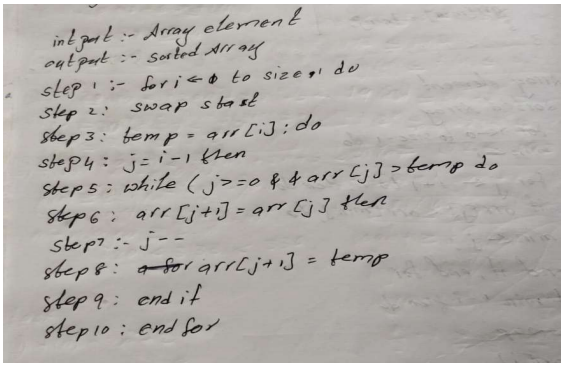
Output:-



Timecomplexity:-



Algorithms:- Insertion sort



Program/code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    // Insertion Sort
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
```

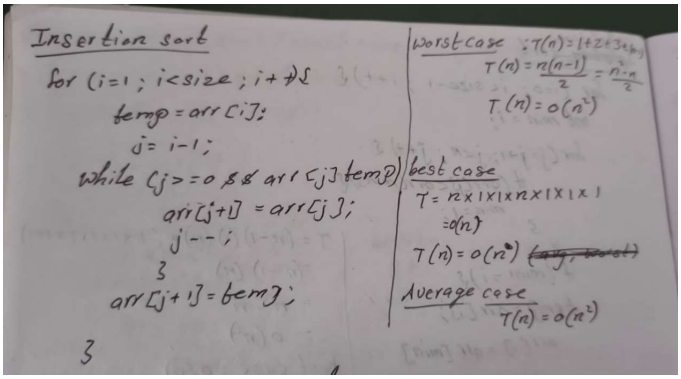
```
        }
        arr[j + 1] = key;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:-

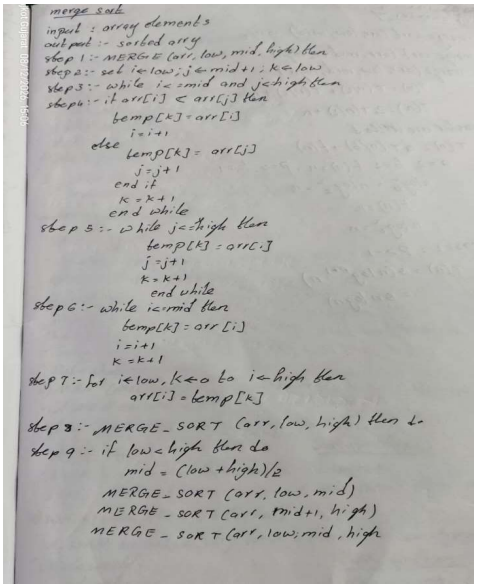
```
Enter the number of elements: 4
Enter the elements: 5
9
7
6
Sorted array: 5 6 7 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Timecomplexity:-



Algorithms:- Merge sort



Program/code:-

```
#include <iostream>
using namespace std;
// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```
// Quick Sort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

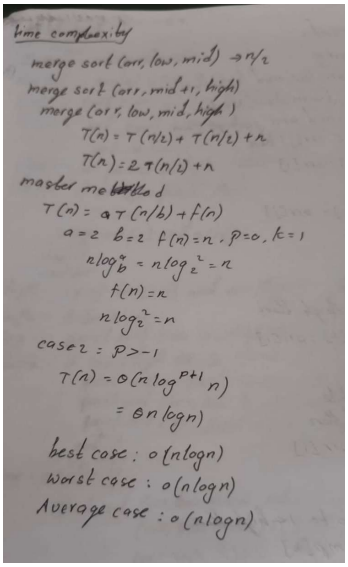
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:-

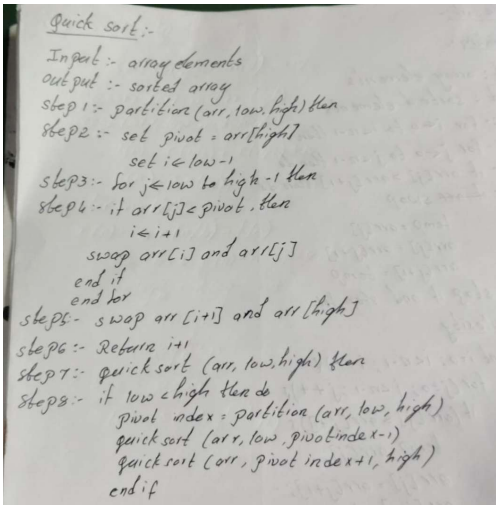
```
Enter the number of elements: 8
Enter the elements: 6
7
2
9
3
1
8
5
Sorted array: 1 2 3 5 6 7 8 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Timecomplexity:-



Algorithms:- Quicksort



Program/code:-

```
#include <iostream>
using namespace std;
// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
// Quick Sort function
```

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

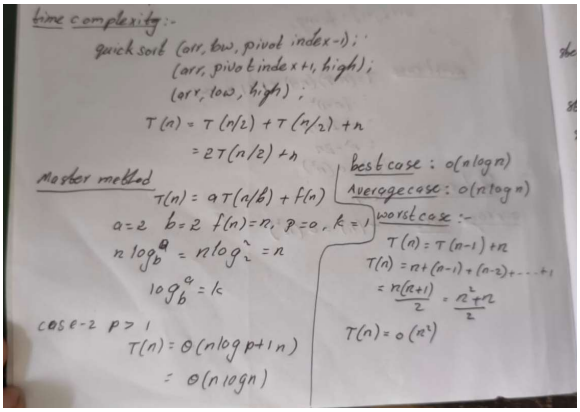
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:-

```
Enter the number of elements: 5
Enter the elements: 9
7
3
1
0
Sorted array: 0 1 3 7 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Timecomplexity:-



Conclusion:-

In conclusion, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort are important sorting algorithms, but their performance differs. Bubble, Selection, and Insertion Sort are simple and easy to implement, but generally take $O(n^2)$ time. Merge Sort provides consistent O(n log n) performance and is suitable for large datasets, while Quick Sort is usually very fast in practice with an average time of O(n log n), although its worst case is $O(n^2)$.

Therefore, Insertion Sort is useful for small or nearly sorted data, Merge Sort is reliable for guaranteed performance, and Quick Sort is often preferred for efficient general-purpose sorting.